Bài 2:

$h_4 = f(x_4, h_3)$

"like"

$h_3 = f(x_3, h_2)$

"not"

Ảnh hưởng:

- RNN: Thông tin "not" trong $h_3$ tương tác với $x_4$ = "like" qua $h_4$, làm đảo chiều vector trạng thái từ tích cực sang tiêu cực
- Avorange W2V: Cộng tb nên "not" k. thể biến đổi hay đảo chiều ngữ nghĩa của "like"

Bài 3:

- Average W2V: Ko phân biệt đc, ưu phép cộng giao hoán cho ra cùng 1 vector tb cho cả 2 câu
- RNN: Cập nhật đệ quy, phép nhân ma trận $W_{hh}$ & hàm phi tuyến tanh lồng nhau qua các bước k. có tính giao hoán dẫn tới $h_s^{(s_1)} \neq h_s^{(s_2)}$

Được quét bằng CamScanner

11 Experiment 1

- Phân vị độ dài trên histogram: chọn quanh vùng phân vị để bao phủ phần lớn mà không bị kéo theo đuôi ngoại lai
- Tỷ lệ token giữ lại: chỉ để giữ lại 21,8% tokens.
- Tỷ lệ lãng phí do padding: là padding ở độ dài cố định tăng vọt từ 3,1% → 20,8%
- Chi phí tính toán: chọn nhỏ của kiến thức: chuỗi quá dài làm tăng thời gian huấn luyện, gây suy giảm đạo hàm

13. Experiment 2

- Tăng max length có tuần cải thiện performance. Tăng từ 50 → 100 giúp F1 tăng mạnh nhưng từ 100 → 200 làm test F1 giảm xuống, Val loss tăng lên vì chuỗi dài 200 bắt chiến Simple RNN bị suy giảm đạo hàm
- Nếu max length quá nhỏ sẽ mất các mối chuyện vì nó phần ở giữa bài, toàn bộ phần nhận xét tổng kết định nhãn xúc ở cuối bài review

Được quét bằng CamScanner

14. Core implementation

- Ở mỗi bước t, input $x_t$ nhân với ma trận trọng số $W_{xh}$ và trạng thái ẩn trước $h_{t-1}$ nhân với ma trận trọng số $W_{hh}$; cộng 2 tích này với bias $b_h$, sau đó đi qua hàm phi tuyến tanh để tạo ra trạng thái mới $h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b_h)$

15. Experiment 3

Có:

- Với các cặp câu có từ trong nhóm ngữ nghĩa thứ tự, áp dụng cho các vector $\Delta L2 = 0$ và độ lệch xs $\Delta P = 0$, mình đã tạo ra ~~xs~~ $\Delta L2$ từ 0,9018 → 1,02714 và thấy đối xs dự đoán.
- Khi hoán vị ngẫu nhiên 100 tokens đầu ở mẫu test, áp dụng cho tất cả dự đoán nào; mình có meanΔPl = 0,1152 và tối nhận phân loại ở mẫu test.

17. LSTM gates

$C_t = (0,9 \times 0,8) + (0,2 \times 0,5) = 0,82$.

- Nếu forget gate gần 1: model có xu hướng giữ lại thông tin cũ từ $C_{t-1}$ truyền sang $C_t$
- Nếu forget gate gần 0: model có xu hướng loại bỏ thông tin cũ ở $C_{t-1}$ để nhường chỗ cho thông tin mới

19. Experiment 4

Phù hợp hoàn toàn

- LSTM và GRU vượt trội hơn Simple RNN: không chỉ tăng acc mà còn cải thiện F1, giảm sâu Val loss và tăng Recall đồng thời giữ chuẩn Gradient ổn định nhờ ~~đóng~~ cơ chế cổng kiểm soát dòng thông tin

CRABIT

Được quét bằng CamScanner

- GRU hiệu quả tương đương nhưng nhanh hơn LSTM: do chỉ có 2 cổng và 1 dòng cell state riêng, GRU ít tham số hơn LSTM giúp tgian huấn luyện ↓ gần 1 nửa và F1 nhỉnh nhẹ hơn.

## 20. Experiments

Có Tăng max_length luôn là tăng chi phí tgian và bộ nhớ GPU. Ở giai đoạn 50 → 100, lg ngữ cảnh tăng thêm giúp F1 tăng mạnh nhất từ 100 → 200 tgian huấn luyện tăng vọt (.) Khi F1 lại giảm do simple RNN ko khai thác hiệu quả dc chuỗi quá dài

## 23. Learning Curves

Có, mô hình đều bị overfitting sau Epoch 2/3
- từ epoch 1 → 2/3: cả train_loss và val_loss cùng giảm
- từ epoch 3/4 → 6: train_loss tiếp tục giảm sâu nhưng val_loss đảo chiều tăng vọt do mô hình học ê lòng tầng embedding trên tập train. Vì vậy p dùng early stopping chọn checkpoint ở epoch 2-3 thay vì epoch cuối

24. Err

25. So sánh với LABOH

- RNN / LSTM / GRU giải quyết đc bài toán thứ tự từ trong thái phụ e ngữ cảnh
- RNN / LSTM / GRU vẫn còn vđề
  +> ko thể tính toán song song theo tgian
  s chi phí cao
  +> Nút thắt cổ chai chi nén toàn bộ chuỗi vào 1 vecto cố định ht

CRABIT

Được quét bằng CamScanner